



Diseño de sistemas en chip (Gpo 603)

Entregable de Reto Final

Profesores:

José Isabel Gómez Quiñones

Rodolfo Anaya Zamora

Socio Formador

Jhon Deere

Estudiantes:

Juan Jose Jauregui Barba- A00836722

Rosendo De Los Rios Moreno - A01198515

Jordan Arturo Palafox Salinas - A00835705

Marcelo Marcos Flores - A01722788

Fecha:

13 de Junio del 2024

ÍNDICE

Introducción.....	3
Marco Teórico.....	4
Sistemas Embebidos.....	4
Sistema en chip.....	4
Microcontrolador.....	4
Aplicaciones de Sistemas Embebidos.....	5
Ventajas de los Sistemas Embebidos.....	5
Desventajas de los Sistemas Embebidos.....	5
Sistemas Operativos en Tiempo Real para Sistemas Embebidos.....	6
Aplicaciones de un RTOS.....	6
Ventajas de un RTOS sobre el diseño de una aplicación bare metal.....	6
Desventajas de un RTOS.....	7
Linux Embebido.....	7
Aplicaciones de Linux Embebido.....	7
Ventajas de Linux Embebido.....	8
Desventajas de Linux Embebido.....	8
Programación con Bash scripts y Python.....	8
Justificación.....	9
Metodología.....	10
Hardware utilizado.....	12
Software utilizado.....	12
Periféricos Utilizados.....	12
Diagrama Esquemático.....	14
Diagrama Esquemático Completo.....	14
Diagrama de interacción de las Tareas (RTOS).....	16
Diagramas de Flujo por Tarea.....	16
Scheduling de Tareas (RTOS).....	16
Acciones de mejora llevadas a cabo para lograr un despliegue gráfico en tiempo real.....	18
¿Por qué es importante el monitoreo en tiempo real dentro del contexto de tu proyecto?.....	18
Descripción de los problemas encontrados.....	18
Contraste de los resultados obtenidos antes y después de las estrategias de mejora.....	19
Api seleccionada.....	20
Descripción de la interfaz gráfica desarrollada en Python para el control de parámetros (torque, freno, aceleración) en el tractor.....	20
Caso de uso con ejemplo de manipulación de torque.....	21
Video caso de uso con ejemplo de manipulación de torque.....	24
Conclusión.....	24
Reflexiones Individuales.....	25
Referencias:.....	29

Introducción

En el ámbito de la agricultura moderna, la integración de tecnologías avanzadas es más que esencial para mejorar la eficiencia y productividad de las operaciones. John Deere, una empresa líder en la manufactura de maquinaria agrícola, se ha asociado con nosotros para desarrollar un sistema inteligente de conducción para tractores. Este proyecto se centra en la conexión de un microcontrolador STM32F103 con una Raspberry Pi, utilizando un acelerómetro para medir la velocidad del motor y gestionar automáticamente la velocidad del tractor, así como el cambio de marcha. Los datos recopilados por el STM32 se muestran en una pantalla LCD y se envían a la Raspberry Pi mediante un módulo FTDI. La Raspberry Pi procesa estos datos y los presenta en forma de gráficos, permitiendo un monitoreo detallado en tiempo real del rendimiento del tractor.

La implementación de un sistema inteligente de conducción en tractores ofrece múltiples beneficios. En primer lugar, mejora la eficiencia operativa al optimizar la velocidad y el cambio de marchas, lo que reduce el consumo de combustible y el desgaste del motor. Además, facilita el trabajo del operador al automatizar tareas complejas, lo que disminuye la probabilidad de errores humanos y mejora la seguridad tanto del tractor como del operador. En un entorno agrícola, donde el tiempo y los recursos son limitados, estas mejoras pueden traducirse en un aumento significativo de la productividad y una reducción de los costos operativos.

Actualmente, existen varias soluciones avanzadas en el mercado para sistemas de conducción en maquinaria agrícola. Entre las más destacadas se encuentran los sistemas de guía automática mediante GPS, que permiten una precisión milimétrica en las operaciones de campo. Además, muchos tractores modernos están equipados con sistemas de telemetría y monitoreo remoto, que permiten a los operadores supervisar el rendimiento de la maquinaria y recibir alertas en tiempo real sobre posibles problemas. Estos sistemas suelen estar integrados con plataformas de gestión agrícola, proporcionando una visión integral de todas las operaciones en curso y permitiendo una toma de decisiones informada basada en datos precisos.

Nuestro proyecto para John Deere busca complementar estas tecnologías existentes, ofreciendo un enfoque adicional que no solo monitorea y visualiza los datos en tiempo real, sino que también ajusta dinámicamente el rendimiento del tractor para optimizar su operación. Al

integrar un acelerómetro con un STM32F103 y una Raspberry Pi, proporcionamos una solución robusta y flexible que puede adaptarse a las necesidades cambiantes de la agricultura moderna.

Con este sistema, esperamos contribuir significativamente a la evolución de la maquinaria agrícola, haciendo que los tractores sean más inteligentes, eficientes y fáciles de operar en un futuro no muy lejano.

Marco Teórico

En el campo de la tecnología moderna, los sistemas embebidos juegan un papel esencial en una variedad de aplicaciones que van desde la electrónica de consumo hasta sistemas industriales complejos. Estos sistemas, que integran hardware y software para realizar funciones específicas dentro de un sistema más amplio, son fundamentales para la eficiencia y funcionalidad de numerosos dispositivos que utilizamos diariamente.

Sistemas Embebidos

Un sistema embebido es una combinación de componentes de hardware y software diseñados para llevar a cabo tareas específicas. Estos sistemas son autónomos y suelen estar integrados dentro de dispositivos más grandes, donde gestionan y controlan operaciones críticas en tiempo real.

Sistema en chip

Un sistema en chip (SoC) es un circuito integrado que combina todos los componentes necesarios para un sistema completo en un solo chip. Estos componentes incluyen una unidad central de procesamiento (CPU), memoria, interfaces de entrada/salida y otros periféricos. Los SoCs son esenciales en dispositivos donde el espacio y la eficiencia energética son críticos, como teléfonos móviles, tablets, dispositivos IoT y sistemas embebidos.

Microcontrolador

Un microcontrolador es un tipo específico de SoC diseñado para controlar dispositivos electrónicos. Integra una CPU, memoria y periféricos en un solo chip y se utiliza en aplicaciones donde la simplicidad, el bajo costo y el bajo consumo de energía son importantes. Los microcontroladores son omnipresentes en la tecnología moderna, desde electrodomésticos y juguetes hasta sistemas automotrices y equipos industriales.

Los SoCs y los microcontroladores facilitan la creación de nuevos productos al proporcionar soluciones compactas y eficientes que pueden integrarse fácilmente en una amplia gama de dispositivos. Su capacidad para manejar tareas específicas de manera eficaz permite a los desarrolladores diseñar productos innovadores y funcionales con menores costos y tiempos de desarrollo reducidos.

Aplicaciones de Sistemas Embebidos

- Automoción: Control de motores, sistemas de frenos antibloqueo (ABS), sistemas de navegación y entretenimiento.
- Electrónica de consumo: Teléfonos móviles, tablets, televisores inteligentes y sistemas de entretenimiento en el hogar.
- Médico: Equipos de diagnóstico, monitores de pacientes y dispositivos de administración de medicamentos.
- Industrial: Controladores lógicos programables (PLC), sistemas de automatización y robótica.

Ventajas de los Sistemas Embebidos

1. Especialización: Diseñados para tareas específicas, lo que aumenta la eficiencia y el rendimiento.
2. Costo operativo bajo: Generalmente tienen un bajo consumo de energía y pueden reducir los costos operativos a largo plazo.
3. Fiabilidad y estabilidad: Su diseño optimizado para tareas específicas suele resultar en una mayor fiabilidad y estabilidad.

Desventajas de los Sistemas Embebidos

1. Flexibilidad limitada: Diseñados para funciones específicas, lo que puede dificultar la adaptación a nuevas tareas.
2. Actualización complicada: Actualizar o mejorar un sistema embebido puede ser complejo y costoso.

3. Ciclo de vida limitado: Algunos sistemas embebidos pueden volverse obsoletos rápidamente debido a la rápida evolución tecnológica.

Sistemas Operativos en Tiempo Real para Sistemas Embebidos

Los sistemas operativos en tiempo real (RTOS) son una pieza clave en el desarrollo de sistemas embebidos, proporcionando una base sólida para aplicaciones donde el tiempo de respuesta es crucial. Un RTOS, o Real-Time Operating System, se diferencia de los sistemas operativos convencionales en su capacidad para garantizar que las tareas se completen en tiempos predecibles, lo que es esencial en aplicaciones que no pueden tolerar retrasos.

Un RTOS es un sistema operativo que proporciona servicios de gestión de tareas y recursos con una garantía de cumplimiento estricto de los plazos temporales establecidos. Estos sistemas operativos se utilizan en aplicaciones críticas como el control industrial, automotriz, aeroespacial y dispositivos médicos.

Aplicaciones de un RTOS

- Automoción: Control de motores, sistemas de frenos antibloqueo (ABS), y sistemas de navegación.
- Aeroespacial: Control de vuelo, sistemas de comunicación y navegación.
- Industrial: Controladores lógicos programables (PLC), sistemas de automatización y robótica.
- Médico: Monitores de pacientes, equipos de diagnóstico y dispositivos de administración de medicamentos.
- Telecomunicaciones: Equipos de red y routers que requieren un procesamiento rápido y fiable.

Ventajas de un RTOS sobre el diseño de una aplicación bare metal

1. Determinismo temporal: Garantiza que las tareas críticas se completen en el tiempo especificado.

2. Gestión eficiente de tareas: Facilita la creación y gestión de múltiples tareas concurrentes.
3. Modularidad y escalabilidad: Permite un diseño más modular y escalable, facilitando el mantenimiento y la ampliación del sistema.
4. Manejo de interrupciones: Proporciona un manejo más eficiente y flexible de las interrupciones.

Desventajas de un RTOS

1. Complejidad: Introduce una capa adicional de complejidad en el diseño y desarrollo del sistema.
2. Consumo de recursos: Puede requerir más recursos de memoria y procesamiento en comparación con una aplicación bare metal.
3. Curva de aprendizaje: Requiere que los desarrolladores se familiaricen con el RTOS y sus características específicas.

Linux Embebido

Linux embebido es una versión del sistema operativo Linux diseñada para funcionar en dispositivos embebidos. A diferencia de las versiones de Linux para computadoras de escritorio o servidores, Linux embebido está optimizado para funcionar con recursos limitados y está adaptado para las necesidades específicas del dispositivo en el que se ejecuta.

Linux se ha adaptado para dispositivos con recursos limitados, como memoria y almacenamiento. Es utilizado en una amplia gama de aplicaciones debido a su flexibilidad y capacidad para ser altamente personalizado.

Aplicaciones de Linux Embebido

- Electrónica de consumo: Teléfonos inteligentes, tablets, televisores inteligentes y sistemas de entretenimiento en el hogar.
- Automoción: Sistemas de infoentretenimiento, navegación y telemetría.

- Industrial: Sistemas de control y automatización, dispositivos de red y telecomunicaciones.
- Dispositivos IoT: Sensores, cámaras de seguridad y dispositivos de hogar inteligente.

Ventajas de Linux Embebido

1. Flexibilidad: Altamente configurable y adaptable a diversas aplicaciones.
2. Código abierto: El acceso al código fuente permite personalizar y optimizar el sistema según las necesidades específicas.
3. Comunidad de soporte: Gran comunidad de desarrolladores y una abundancia de recursos y documentación.
4. Compatibilidad: Soporta una amplia gama de hardware y periféricos.

Desventajas de Linux Embebido

1. Curva de aprendizaje: Puede ser complicado de configurar y optimizar para desarrolladores no familiarizados con Linux.
2. Consumo de recursos: Puede requerir más recursos que otros sistemas operativos embebidos más ligeros.
3. Latencia no determinista: No siempre garantiza tiempos de respuesta estrictos en aplicaciones de tiempo real sin modificaciones específicas.

Programación con Bash scripts y Python

- Bash scripts: Bash es el intérprete de comandos predeterminado en muchas distribuciones de Linux. Los scripts de Bash permiten automatizar tareas repetitivas y manejar operaciones del sistema de manera eficiente. Son especialmente útiles para tareas de administración del sistema, gestión de archivos y configuración de entornos.
- Python: Python es un lenguaje de programación de alto nivel conocido por su legibilidad y facilidad de uso. En el contexto de sistemas embebidos, Python es valioso por su amplia gama de bibliotecas y su capacidad para realizar tareas complejas de

manera sencilla. Se utiliza para el desarrollo rápido de prototipos, automatización de tareas y manejo de datos.

Justificación

La modernización de la agricultura mediante la integración de tecnologías avanzadas es una necesidad hoy en día para aumentar la eficiencia, la productividad y la sostenibilidad. En este contexto, el proyecto desarrollado en colaboración con John Deere, combina un microcontrolador STM32F103 y una Raspberry Pi, representa un proyecto importante hacia la optimización del rendimiento de los tractores agrícolas

El microcontrolador STM32F103 fue seleccionado para este proyecto debido a su capacidad de manejar tareas en tiempo real con alta precisión. Este dispositivo actúa como el núcleo del sistema, procesando datos de un acelerómetro para controlar la velocidad del motor y gestionar los cambios de marcha del tractor. La precisión y capacidad de respuesta del STM32F103 fueron uno de los principales motivos para asegurar que el tractor funcione de manera óptima, ajustando su velocidad y cambio de marchas en función de las condiciones operativas. Esto no solo mejora la eficiencia del tractor, sino que también contribuye a la seguridad y la reducción del desgaste del equipo.

La utilización de un Sistema Operativo en Tiempo Real (RTOS) en el STM32F103 garantiza una gestión eficiente y precisa de las tareas críticas. Un RTOS proporciona control sobre los tiempos de ejecución de las tareas, lo cual es crucial en aplicaciones donde la sincronización y la respuesta rápida son vitales. En este proyecto, los RTOS permiten que el microcontrolador responda inmediatamente a los cambios en los datos del acelerómetro, ajustando la velocidad del motor y los cambios de marcha de manera oportuna. Esto asegura que el tractor opere bajo las mejores condiciones posibles, optimizando su rendimiento y vida útil.

La inclusión de la Raspberry Pi en el proyecto añade es necesaria por su capacidad de procesamiento y flexibilidad. La Raspberry Pi, con su sistema operativo Linux embebido, se encarga de recibir los datos transmitidos por el STM32F103 y procesarlos para su análisis y visualización en tiempo real. La capacidad de Raspberry Pi para ejecutar scripts de Bash y aplicaciones en Python facilita la automatización de procesos y el manejo eficiente de los datos. Esto permite una visualización detallada de la velocidad del motor, la velocidad del tractor y

los cambios de marcha, proporcionando una plataforma para análisis más complejos y generación de informes que pueden utilizarse para mejorar las operaciones agrícolas.

Este proyecto se desarrolla para demostrar la aplicabilidad y las ventajas de los sistemas embebidos en la agricultura, al mismo tiempo también subraya la importancia de la tecnología en la modernización y optimización de las operaciones agrícolas. Al integrar el microcontrolador STM32F103 y la Raspberry Pi, se crea un sistema robusto y escalable que puede adaptarse a futuras innovaciones y mejoras. Esto es particularmente relevante en la industria agrícola, donde la eficiencia y la capacidad de respuesta son cruciales para mantener la competitividad y satisfacer las demandas de producción.

Además, este proyecto busca reforzar el compromiso de John Deere con la innovación y la excelencia. La implementación de este sistema embebido no solo mejora la eficiencia operativa del tractor, sino que también proporciona una plataforma flexible que puede adaptarse a futuras mejoras tecnológicas. Esto asegura que John Deere continúe liderando la industria agrícola, ofreciendo soluciones avanzadas que benefician tanto a los agricultores como al medio ambiente.

Metodología

El desarrollo del proyecto con John Deere se centró en la creación de un sistema embebido que integra un microcontrolador STM32F103 y una Raspberry Pi para medir, controlar y visualizar la velocidad del motor y los cambios de marcha de un tractor. Este esfuerzo abarcó desde la conceptualización y el prototipado hasta la implementación final, destacando el uso de tecnologías avanzadas para mejorar la eficiencia y el rendimiento de maquinaria agrícola.

El primer paso del proyecto fue la conceptualización, donde se definieron los objetivos principales y se identificaron las necesidades específicas del sistema. El STM32F103 fue seleccionado debido a su capacidad para manejar tareas de control en tiempo real, mientras que la Raspberry Pi se eligió por su versatilidad y potencia de procesamiento, ideal para el análisis y la visualización de datos. Se decidió utilizar un acelerómetro para medir la velocidad del motor, un LCD para la visualización local y un módulo FTDI para la comunicación serial entre el microcontrolador y la Raspberry Pi.

En la fase de prototipado, se desarrolló un diseño preliminar que incluía la selección de componentes clave y la construcción de circuitos de prueba donde se encontraba la conexión de la STM32F103, la conexión del acelerómetro y el LCD. Se implementaron scripts básicos en Raspberry Pi para asegurar la correcta recepción y visualización de datos. Estos pasos fueron esenciales para validar la interacción entre el STM32F103 y los periféricos seleccionados, y para garantizar que el sistema pudiera cumplir con los requisitos establecidos.

La implementación final involucró la integración de todos los componentes y subsistemas. El microcontrolador STM32F103 se programó en C/C++ utilizando la plataforma STM32Cube IDE. Este microcontrolador se encargó de leer los datos del acelerómetro, calcular la velocidad del motor y gestionar los cambios de marcha. Los datos obtenidos se mostraron en un LCD y se transmitieron a la Raspberry Pi a través del módulo FTDI. En la Raspberry Pi, se utilizó Raspbian OS y Thonny IDE para desarrollar aplicaciones en Python que procesaban los datos recibidos y generaban gráficos en tiempo real.

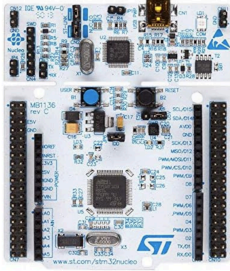
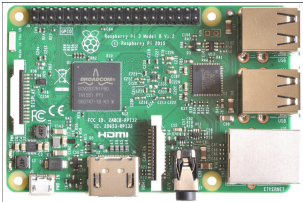
El hardware utilizado en este proyecto incluyó tarjetas de desarrollo y componentes específicos como el STM32F103, la Raspberry Pi, el acelerómetro, el LCD y el módulo FTDI. Adicionalmente, la programación del STM32F103 aprovechó recursos críticos como el ADC para leer los datos del acelerómetro, el UART para la comunicación serial y los GPIO para el control del LCD. Por su parte, la Raspberry Pi utilizó la FTDI para recibir datos del microcontrolador, y las bibliotecas de Python, como matplotlib y pandas, para procesar y visualizar los datos.

La interacción entre el STM32F103 y la Raspberry Pi fue un aspecto crucial del proyecto. El STM32F103 realizaba las mediciones y cálculos necesarios y transmitía los datos a la Raspberry Pi, que se encarga del procesamiento y la visualización detallada de los mismos. Esta configuración permite una supervisión constante del desempeño del tractor y una respuesta rápida a cualquier cambio en las condiciones de operación.

A lo largo del proyecto, se llevaron a cabo múltiples iteraciones de pruebas y optimización para asegurar la precisión y fiabilidad del sistema. Estas pruebas incluyen la calibración del acelerómetro, el ajuste de los parámetros de control del motor y la validación de la transmisión de datos entre el STM32F103 y la Raspberry Pi. Las pruebas en condiciones reales permitieron

ajustar el sistema para lograr un desempeño óptimo, lo que fue esencial para cumplir con los estándares de calidad y eficiencia requeridos por John Deere.

Hardware utilizado

Nombre del Componente	Componente usado
STM32F103RB MCU	
Raspberry Pi 3 Modelo B+ (3 B Plus)	

Software utilizado

Nombre del Software
STM32CubeIDE
Raspberry Pi image
Visual Studio Code
PUTTYGen
CMSIS

Periféricos Utilizados

Nombre del Componente	Componente usado
-----------------------	------------------

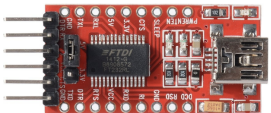
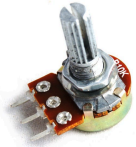
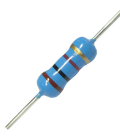
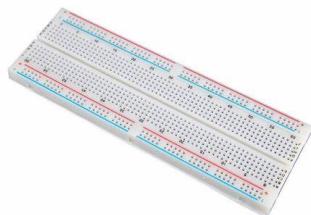
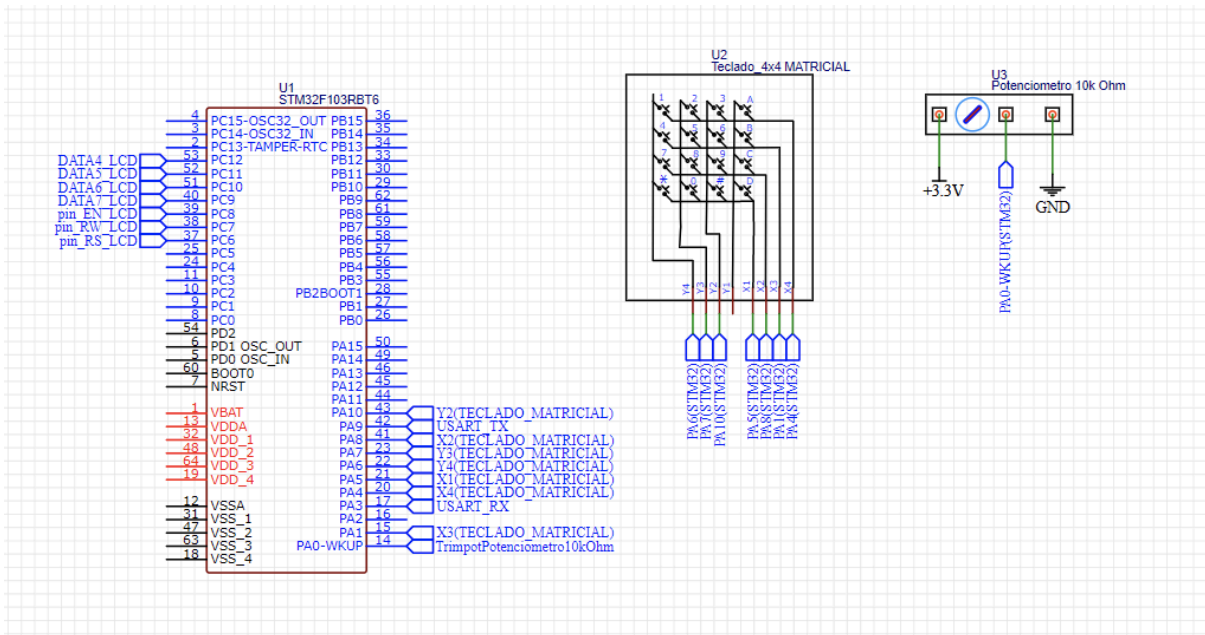
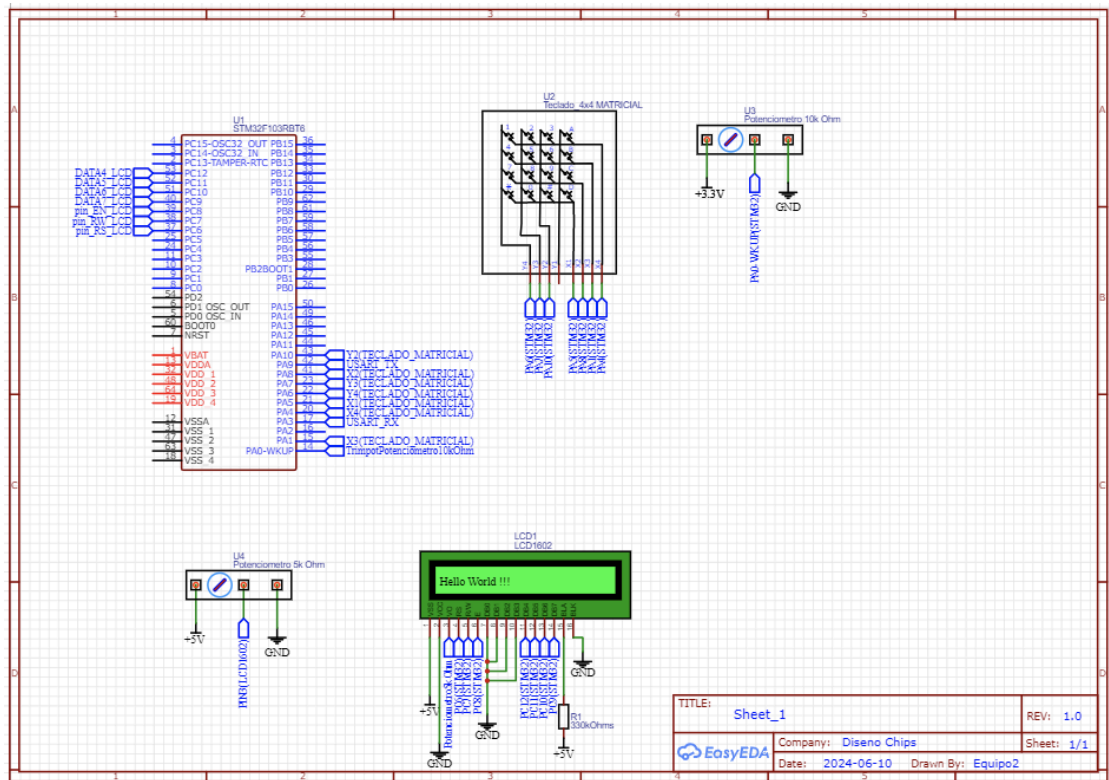
FT232RL FTDI Mini USB	
LCD 16x2 Display VA White	
Potenciómetro Giratorio Poti lineal B10K	
Potenciómetro Giratorio Poti lineal B5K	
Resistencia 330k Ohms	
Jumper de 20 cms con conector Dupont de 2.54mm para Arduino	
Protoboard de ensamble a presión, 1 bloque y 2 tiras	
16 Key 4x4 Matrix Switch Keyboard	

Diagrama Esquemático

Diagrama Esquemático Completo



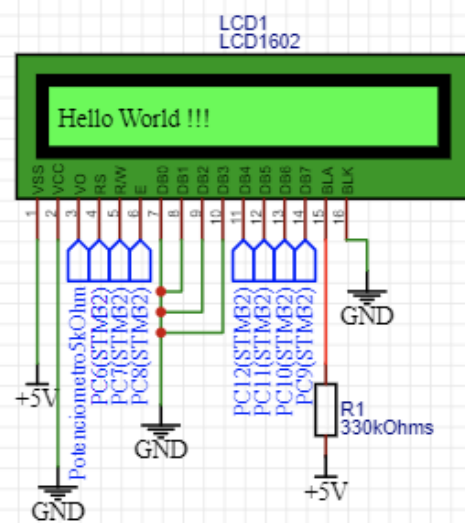
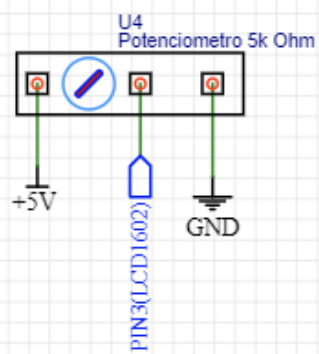
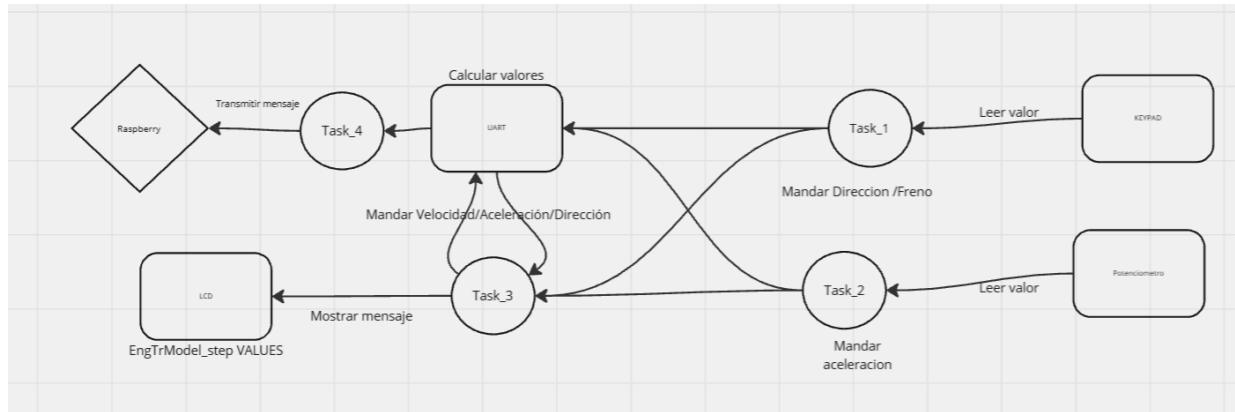
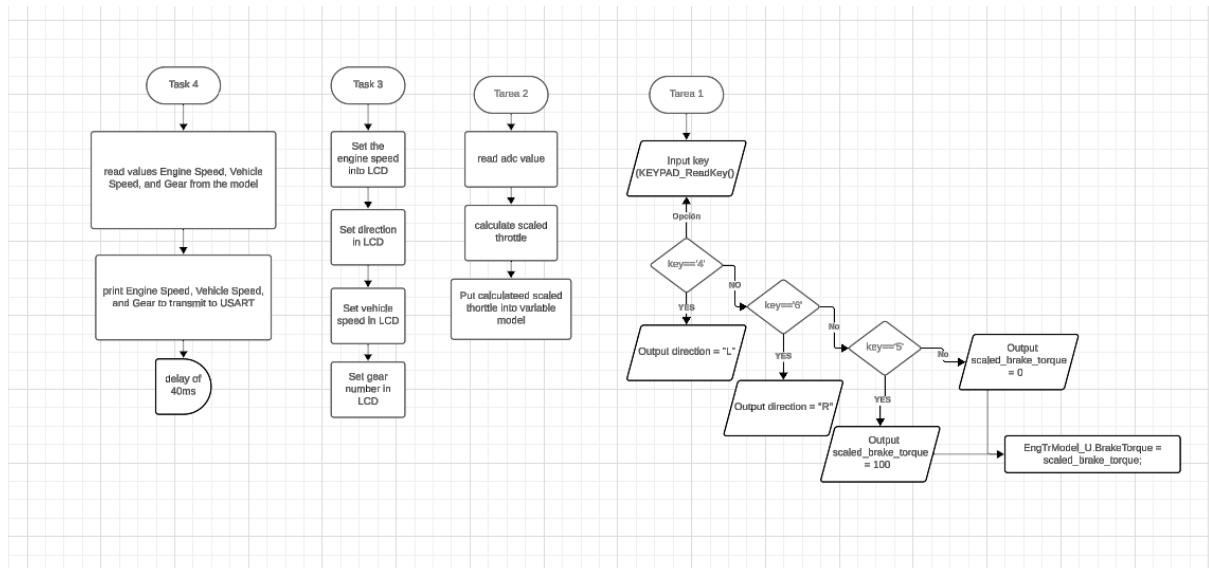


Diagrama de interacción de las Tareas (RTOS)



Diagramas de Flujo por Tarea



Scheduling de Tareas (RTOS)

Tabla con el set de Tareas y sus características

Funcion	Descripción	Dependencias
Task 1 Task1_Init	Inicializa y lee el keypad. Determina la dirección basada en la tecla presionada. Escala el valor de la tecla para el freno.	KEYPAD_Init, KEYPAD_ReadKey
Task 2 Task 2Init	Inicializa y lee el valor del ADC (Potenciometro). Escala el valor del	USER_ADC1_Init, USER_ADC1_Read, EngTrModel_U

	acelerómetro y actualiza el USER_ADC1_Init, USER_ADC1_Read, EngTrModel_U modelo de tren motriz.	
Task 3 Task3_Init	Se inicia y actualiza la pantalla LCD con información sobre la velocidad del motor, la dirección y la velocidad del vehículo.	LCD_Init, LCD_Clear, LCD_Set_Cursor, LCD_Put_Str, LCD_Put_Num
Task 4 Task4_Init	Inicializa y transmite la información a través del puerto serie USART1. Actualiza el modelo de tren motriz y transmite los valores leídos	USER_USART1_Init, USER_USART1_Transmit

Task Set (Utilization: 75.48%)

Task ID	Period	Duration
4	50	1
5	50	2
6	118	48
7	250	72

RMS Schedule

T4 ₀	T5 ₀	T6 ₀														
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	
33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	
		T4 ₁	T5 ₁	T7 ₀												
49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	

65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96
97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112
113	114	115	116	117	118	119	120	121	122	123	124	125			

Acciones de mejora llevadas a cabo para lograr un despliegue gráfico en tiempo real

¿Por qué es importante el monitoreo en tiempo real dentro del contexto de tu proyecto?

La necesidad de mejora en el sistema de monitoreo y control en tiempo real para tractores es esencial, esto debido a que la latencia actual de transmisión y procesamiento de datos es demasiado alta causando un tiempo de espera extenso el cual no es viable para la graficación de datos. Para aplicaciones como el sembrado y la cosecha, que requieren respuestas inmediatas y precisas, este retraso es inaceptable, y dentro del contexto del proyecto con colaboración con Jhon Deere esto afecta de manera negativa la operatividad y la eficiencia del tractor, ya que tanto para el conductor así como para quien visualice los datos, afectando en eficiencia de cosecha y así mismo causando una pérdida económica. Por lo tanto, la optimización de la raspberry pi, que incluye el uso de librerías más eficientes, la eliminación de tareas innecesarias y uso de hilos en el código para ejecutar las tareas en paralelo son esenciales para garantizar que los datos se procesen y se muestran en tiempo real, esto para mejorar la eficiencia, la seguridad y el rendimiento del vehículo en el que se trabaja, también abordar estos desafíos técnicos importante para la mejora y la productividad en la industria agrícola.

Descripción de los problemas encontrados

El monitoreo y graficación de valores en tiempo real presenta múltiples desafíos técnicos y prácticos. Entre los problemas encontramos dificultades con la latencia y velocidad de transmisión, donde la STM32F103 enviaba datos a la Raspberry Pi, la transmisión de datos tomaba aproximadamente un minuto de retraso entre la tarjeta STM32F103 con la conexión UART de esta hacia la Raspberry Pi, esto demuestra que lo anterior dicho pudo haber causado

retrasos para visualizarlos en tiempo real con posibles problemas en el uso de varias herramientas de software y hardware que utilizamos. En cuanto a la Raspberry, era la causante de los problemas en la velocidad de transmisión de datos, ya que esta es una computadora de placa reducida diseñada para ser económica y accesible, por esto se debe optimizar un procesamiento eficiente en el código, al ajustar la frecuencia de actualización de la gráfica, el uso de hilos para separar las tareas, el uso de librerías más eficientes y la reducción de otros programas innecesarios que usan los recursos de la Raspberry Pi son otros problemas a considerar, esto para que tenga suficiente memoria y CPU asegurar que los datos se fueran transmitidos y procesados con la suficiente velocidad para ser útiles en un entorno de tiempo real, sin embargo experimentamos con una latencia de atraso de alrededor de un minuto, lo cual era bastante elevada para un sistema de monitoreo en tiempo real para un tractor. Por lo tanto, uno de los mayores factores de atraso fue el BaudRate bajo, el exceso de datos procesados a la hora de graficar, no considerar un sistema que permita realizar procesos de forma paralela otro factor influyente se encuentran librerías las cuales causaban problemas en el procesamiento de datos en tiempo real ya que eran innecesarias o con demasiadas funciones dentro de su código.

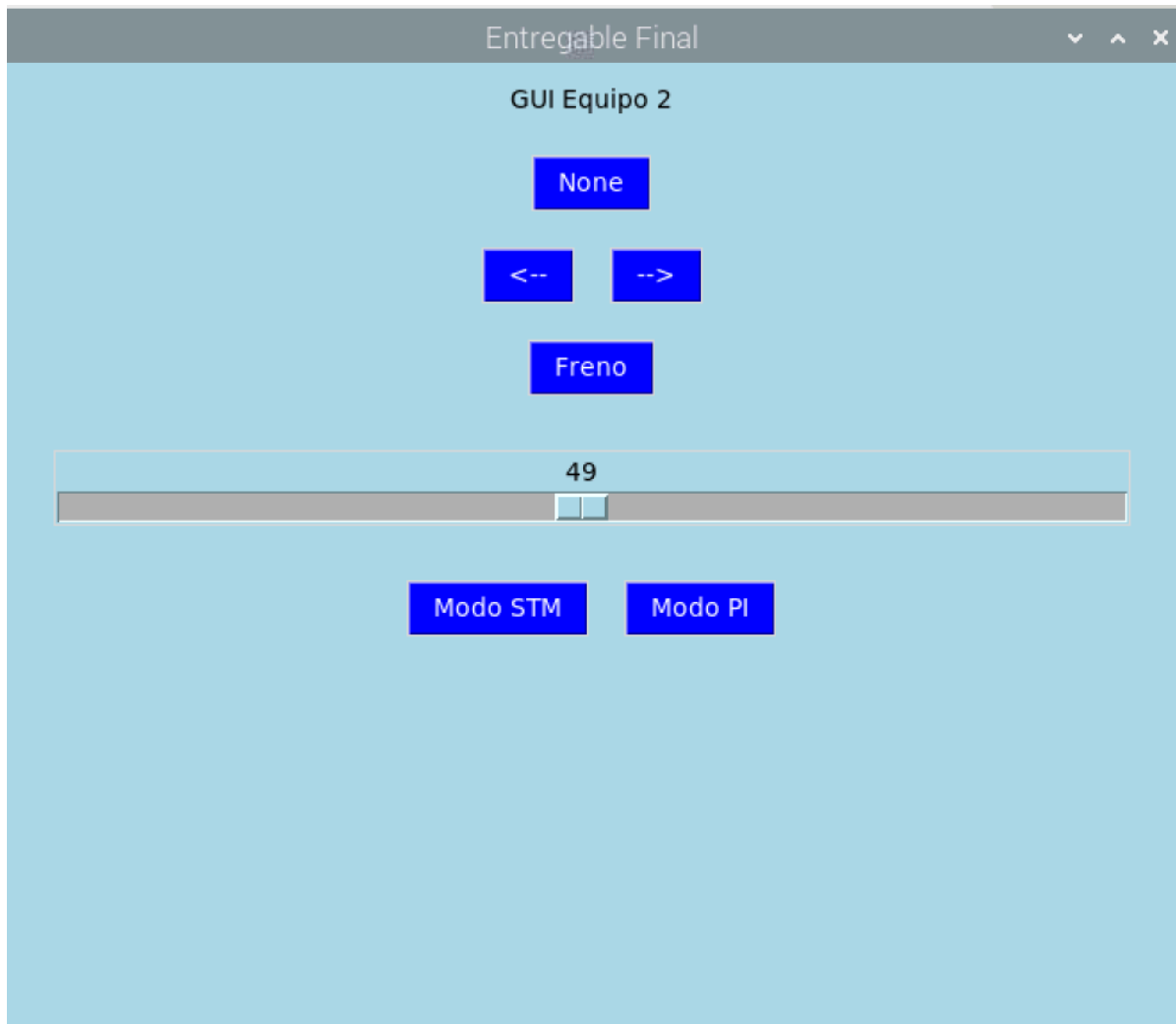
Contraste de los resultados obtenidos antes y después de las estrategias de mejora.

Para la solución de los problemas encontrados se implementó una técnica centrada en la optimización del procesamiento y la visualización de datos para resolver el problema de la latencia en el sistema de monitoreo y control en tiempo real. Inicialmente se usaba PyQtGraph en lugar de Matplotlib, ya que es más eficiente y está diseñado para manejar gráficos en tiempo real con menos recursos de procesamiento. Esta librería permite la actualización rápida y fluida de los gráficos, lo cual es esencial para reducir la latencia y mejorar la respuesta del sistema. Además, optimización en el código utilizando hilos, esto para permitir el procesamiento paralelo de tareas. Esto garantizo que la recopilación, transmisión y visualización de datos se realizarán de manera simultánea y sin interrupciones

En el presente video se muestran los resultados obtenidos antes y después de las estrategias de mejora, el motivo del video es debido a que en fotos no se aprecia correctamente el envío de datos atrasado y el envío de datos de manera actual y mejorada.

Link: [Entrega3.mp4 - Google Drive](#)

Interfaz gráfica desarrollada en Python



Api seleccionada

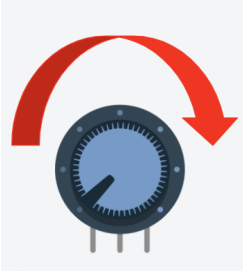
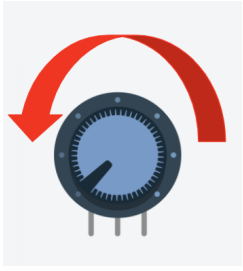
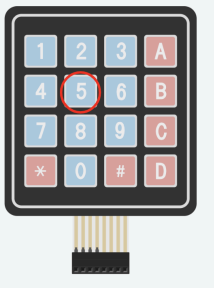
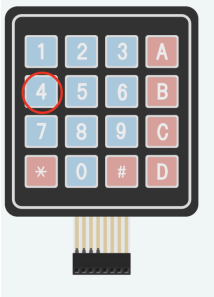
Como apis usamos 2, primero PyQtGraph para poder graficar los datos que envía la STM32 en tiempo real, la segunda Api es Tkinter y la usamos para poder crear una interfaz gráfica en la raspberry pi y poder crear botones para tener un control y mandar datos de la raspberry pi a la STM32.

Descripción de la interfaz gráfica desarrollada en Python para el control de parámetros (torque, freno, aceleración) en el tractor.

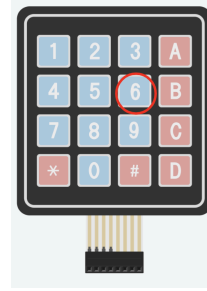
Primero tenemos un botón de None para cambiar el estado de la variable que controla a los botones de girar a la izquierda y derecha, esto porque una vez que pulso el botón girar derecha, va a mandar "R" hasta que presiones otro botón para cambiarlo. Luego está el freno para poder frenar. Luego la barra

deslizable que tenemos como si fuera un potenciómetro para poder modificar los valores de velocidad. Por último están los botones de “Modo STM” para sólo recibir los valores de la STM32 y graficarlos en pyqtgraph, el otro botón “Modo PI” es para parar de recibir esos valores y poder mandar los nuevos valores que mandamos con la barra deslizable y los manda a la STM32.

Caso de uso con ejemplo de manipulación de torque

Manipulación de Torque con STM32F103	Descripción Gráfica
Aumento de velocidad de motor	
Decremento de velocidad de motor	
Frenado del tractor	
Dirección izquierda del tractor	

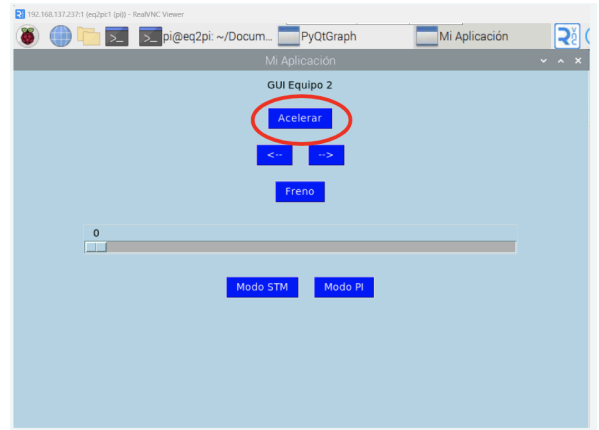
Dirección derecha del tractor



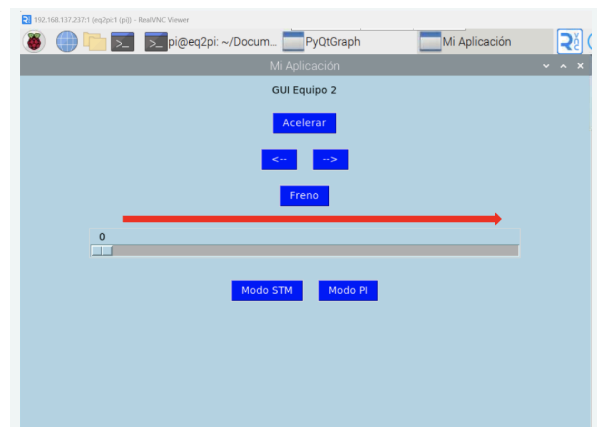
Manipulación de Torque con Raspberry Pi

Descripción Gráfica

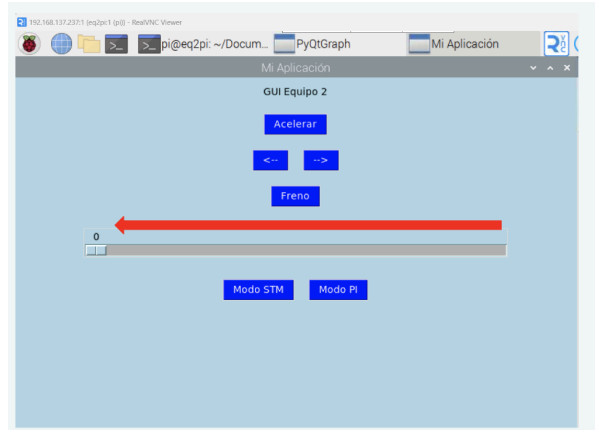
Aceleración del tractor



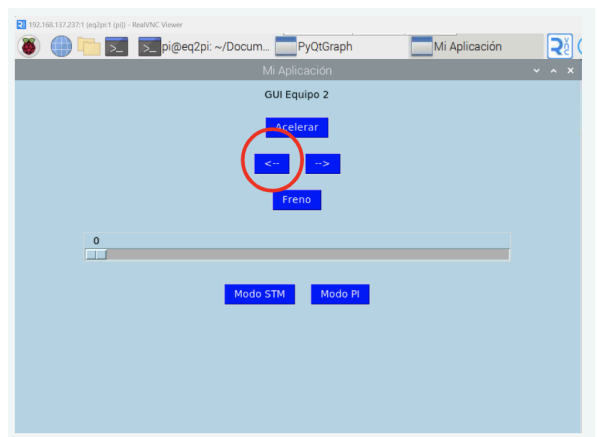
Incremento de velocidad de motor



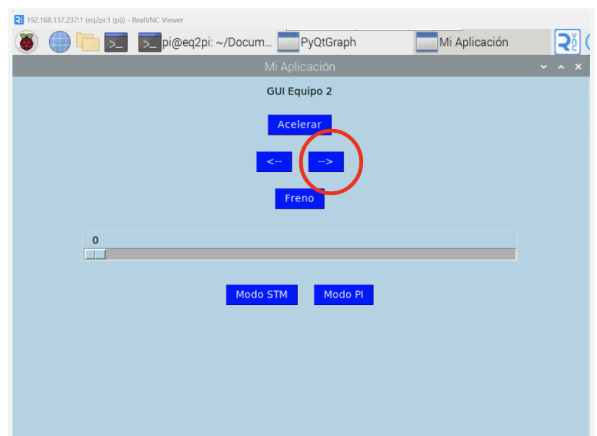
Decremento de velocidad de motor

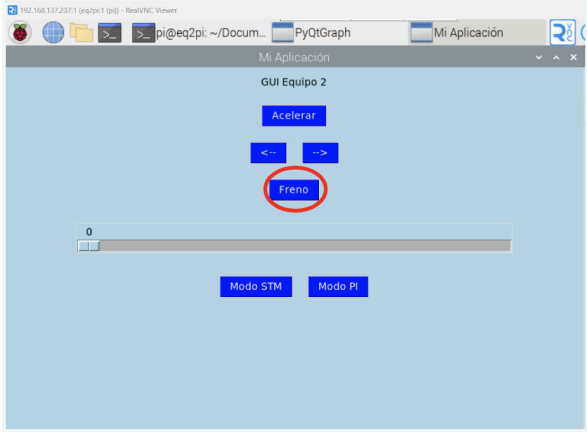
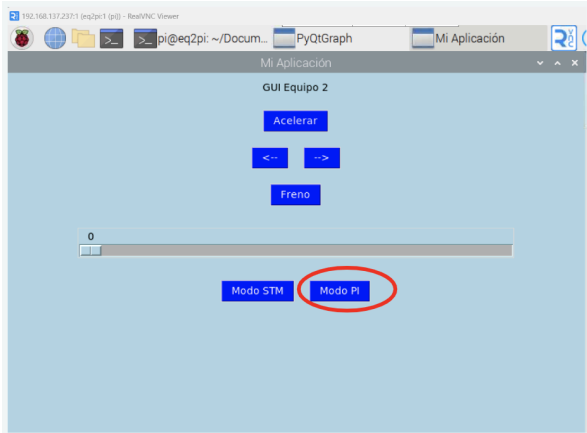
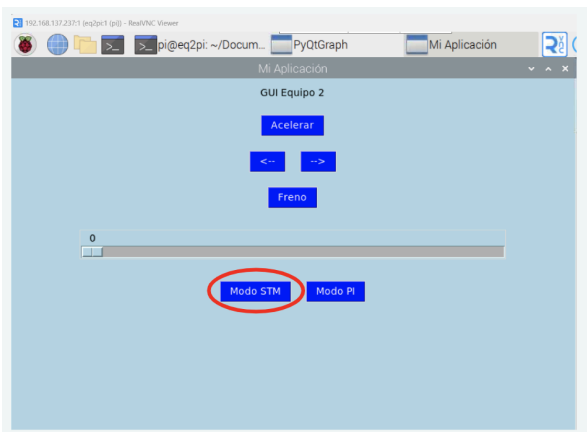


Direccion izquierda del tractor



Direccion derecha del tractor



Freno del Tractor	 The screenshot shows a web-based GUI titled 'Mi Aplicación' running in a browser. It features a light blue background with several control elements: a blue 'Acelerar' button at the top, two blue buttons with left and right arrows below it, and a blue 'Freno' button circled in red. Below these buttons is a horizontal slider bar with a value of 0 on the left. At the bottom, there are two blue buttons labeled 'Modo STM' and 'Modo PI'.
Cambio de interacción Manipulación Raspberry Pi	 This screenshot is similar to the first one, but the 'Freno' button is no longer circled. Instead, the 'Modo PI' button at the bottom right is circled in red. The other UI elements, including the 'Acelerar' button, arrows, and slider, remain the same.
Cambio de interacción Manipulación STM32F103	 In this screenshot, the 'Modo STM' button at the bottom left is circled in red. The 'Modo PI' button is no longer circled. All other UI elements are consistent with the previous two screenshots.

Video caso de uso con ejemplo de manipulación de torque

Video: [Video caso de uso con ejemplo de manipulación de torque.MOV](#)

Conclusión

El proyecto desarrollado en colaboración con John Deere demuestra el potencial transformador de los sistemas embebidos en la agricultura moderna. Al combinar la precisión y capacidad de

respuesta del microcontrolador STM32F103 con la versatilidad y poder de procesamiento de la Raspberry Pi, se ha creado un sistema robusto y eficiente para el control y monitoreo de la velocidad del motor y los cambios de marcha de un tractor. Este sistema no solo mejora la eficiencia operativa, sino que también proporciona una plataforma flexible para futuras innovaciones.

La integración de un RTOS en el microcontrolador asegura una gestión precisa y eficiente de las tareas críticas, mientras que Linux embebido en la Raspberry Pi facilita el procesamiento y visualización de datos en tiempo real. Este enfoque permite un monitoreo continuo y una respuesta rápida a los cambios en las condiciones operativas, mejorando tanto la seguridad como la productividad del tractor.

La concentración de este proyecto demuestra la importancia de la tecnología en la modernización y optimización de las operaciones agrícolas. La capacidad de adaptar y escalar el sistema con comunicación inalámbrica y técnicas avanzadas de análisis de datos posiciona a este proyecto como algo innovador en cuanto a la agricultura de precisión. La implementación a futuro de comunicación inalámbrica y algoritmos de aprendizaje automático no sólo ofrecerá una precisión aún mayor, sino que también permitirá la detección temprana de posibles fallos y la optimización del rendimiento del tractor en diversas condiciones de operación.

Este proyecto destaca la capacidad de los sistemas embebidos para transformar las prácticas agrícolas, incrementando la eficiencia y la sostenibilidad. Además, refuerza el compromiso del los estudiantes del equipo 2 de el Tecnológico de Monterrey y de John Deere con la innovación y la excelencia, proporcionando una base sólida para futuras mejoras tecnológicas. La integración de estas tecnologías avanzadas contribuye significativamente a la competitividad y sostenibilidad de las operaciones agrícolas, demostrando cómo la tecnología puede beneficiar tanto a los agricultores como al medio ambiente.

Reflexiones Individuales

Rosendo De Los Ríos: Trabajar en este proyecto ha sido todo un reto desde mi perspectiva ya basándome y enfocándome más en lo que es mi carrera de ingeniería en robótica y sistemas digitales. Integrar un microcontrolador STM32F103 y una Raspberry Pi me permitió entender

cómo los sistemas embebidos pueden revolucionar no sólo la agricultura sino muchos otros campos. Aprendí a gestionar tareas críticas en tiempo real utilizando un RTOS, lo cual es fundamental para garantizar la precisión y eficiencia del sistema en aplicaciones dinámicas y se podrá usar en un futuro a la hora de gestionar tareas de prioridades en proyectos de mayor peso.

La configuración de Linux embebido en la Raspberry Pi amplió me hizo retomar cursos y aprendizajes pasados mediante la programación, especialmente en Python y Bash scripts, facilitando el manejo y análisis avanzado de datos. Esta experiencia destacó la importancia de la integración de múltiples tecnologías y disciplinas para crear soluciones tecnológicas robustas y eficaces. Para este proyecto no solo considero que fortaleció mis habilidades técnicas, sino que también aprendí cómo la tecnología puede potenciar múltiples ramas donde se usen los micro controladores, en este caso la agricultura hacia la eficiencia y sostenibilidad.

Marcelo Marcos Flores: Este proyecto fue una experiencia fundamental para lograr aprender cómo funciona el manejo de chips STM32 integrado con varios componentes como el FTDI, UART, KEYPAD, LCD y Potenciómetro. Aquí vimos maneras de configurar el chip STM32 para lograr usar estos componentes e integrarlos para obtener valores, pero además logramos la interconexión de estos componentes usando el sistema de RTOS implementada de una manera efectiva con varias tareas para optimizar y manipular las tareas a nuestra necesidad con los componentes y el modelo del tractor que teníamos a nuestra disposición. Adicionalmente aprendimos a recibir y transmitir datos en la STM32 via UART donde se conectaba con una Raspberry PI. Esto implica otra herramienta que desarrollamos y experimentamos para hacer uso de transmisión a tiempo real con Linux.

Este proyecto fue una manera efectiva de implementar un sistema embebido para un uso de varios componentes e interconexiones con programas complejos para el uso de manejo de la optimización de tareas en RTOS y el manejo de datos a través de hardware y software con la interfaz efectiva en estos sistemas embebidos. Este proyecto es un paso para la implementación de sistemas embebidos con varias herramientas importantes a desarrollar que forman parte de mi desarrollo profesional.

Juan Jose Jauregui Barba: Participar en este proyecto ha sido una experiencia transformadora en mi trayectoria como estudiante de robótica y sistemas digitales. Una de las tareas más desafiantes para mí fue la integración de un microcontrolador con una Raspberry Pi.

Por parte de la raspberry pi me gusto mucho aprender a como programar con códigos de python y con bash scripts para poder graficar datos en tiempo real con librerías de python. También me gustó aprender sobre los comandos de Linux, ya que estos los voy a usar seguramente cuando queramos aprender sobre ROS. Lo que también me gusto mucho fue aprender a como crear una interfaz gráfica con tkinter, esta es una librería muy útil para proyectos como estos en los que se tengan que crear una GUI. Por último me gusto como logramos implementar la raspberry pi con la STM32 para estar mandando y recibiendo datos mutuamente en tiempo real.

Además, el proceso de aprender a configurar y utilizar la Raspberry Pi como una herramienta central en el proyecto me enseñó mucho sobre la integración de hardware y software. Por ejemplo, configurar la red y los periféricos, y asegurarse de que la Raspberry Pi pudiera comunicarse eficientemente con otros dispositivos, fue un desafío que me permitió desarrollar habilidades prácticas en redes y conectividad.

Jordan Arturo Palafox: Trabajar en este proyecto fue todo un trabajo extenso para mí. Usar el microcontrolador y la Raspberry Pi fue algo muy interesante pero al mismo tiempo un desafío. Aprendí sobre los múltiples sistemas embebidos, al igual me llevo como aprendizaje como estos se usan en distintos ambientes laborales e industrias. Lo más interesante fue usar los RTOS para manejar tareas en tiempo real. Al principio, fue complicado entender, pero ver cómo todo va funcionando con precisión fue increíblemente gratificante. Esta experiencia me ha enseñado la importancia de la eficiencia y la gestión de prioridades, y estoy seguro de que estos conocimientos serán muy valiosos en mis futuros proyectos.

Trabajo a Futuro

Como visión a futuro de nuestro proyecto se busca implementar varias estrategias que incrementarán su eficiencia, funcionalidad y escalabilidad. En primer lugar, la actualización del software también es crucial. La adopción de técnicas de aprendizaje automático y análisis predictivo en la Raspberry Pi puede ayudar a anticipar y prevenir fallos mecánicos, optimizar el rendimiento del motor y ajustar automáticamente las configuraciones del tractor en respuesta a las condiciones del terreno y clima.

Además, el desarrollo de una interfaz de usuario más intuitiva y accesible facilitará el uso del sistema por parte de operadores con distintos niveles de experiencia tecnológica, incluyendo la

creación de aplicaciones móviles que permitan monitorear y controlar el tractor desde cualquier lugar.

Finalmente, la implementación de un sistema de mantenimiento predictivo, basado en la recopilación y análisis continuo de datos operativos, puede ayudar a reducir el tiempo de inactividad del tractor y prolongar su vida útil. Estas mejoras no solo incrementarían la eficiencia del sistema, sino que también alinearán el proyecto con las tendencias actuales de la agricultura de precisión y la automatización inteligente.

Referencias:

JL, B. (2024, March 4). *Sistema embebido*. Electrónica Online.

<https://electronicaonline.net/electronica/sistemas-embebidos/>

Moraguez, E. R. (2023, March 27). *¿Qué es un Microcontrolador: cómo funciona y para qué sirve?* | LovTechnology. LovTechnology.

<https://lovtechnology.com/que-es-un-microcontrolador-como-funciona-y-para-que-sirve/>

Isaac. (2022, January 21). RTOS: qué es un sistema operativo de tiempo real. *Hardware Libre*.

<https://www.hwlibre.com/rtos/>

Trbl. (2024, January 25). *LINUX Embebido | Qué es, cómo funciona y para qué se usa*. TRBL Services. <https://trbl-services.eu/blog-linux-embebido-que-es/>

Bash Script - Guía completa con ejemplos. (2024, May 3). *Blog HostingTG*.

<https://www.hostingtg.com/blog/bash-script/>

STMicroelectronics. (n.d.). *STMicroelectronics: Our technology starts with you*.

https://www.st.com/content/st_com/en.html

ELinux.org. (n.d.). https://elinux.org/Main_Page

The Yocto Project. (2024, May 31). *The Yocto Project*. <https://www.yoctoproject.org/>

Raspberry Pi Foundation. (n.d.). *Teach, learn, and make with the Raspberry Pi Foundation*.

<https://www.raspberrypi.org/>

Isaac. (2022, January 21). RTOS: qué es un sistema operativo de tiempo real. *Hardware Libre*.

<https://www.hwlibre.com/rtos/>